

Anlea Maria Jose, SE AIML, Roll No.: 02,

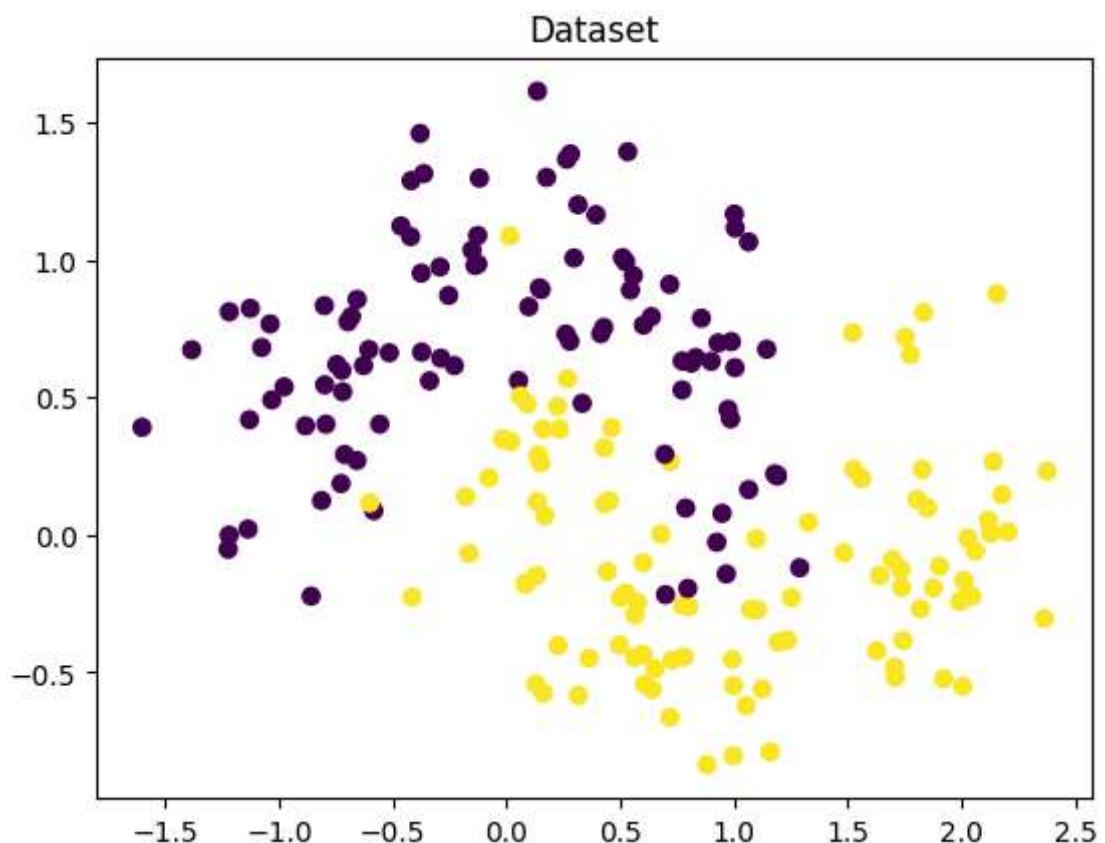
Date: 4-02-2026

```
In [2]: import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix

#Create 2D dataset
X, y = make_moons(n_samples=200, noise=0.25, random_state=42)

plt.scatter(X[:,0], X[:,1], c=y)
plt.title("Dataset")
plt.show()
```



```
In [15]: #Train-Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.3, random_state = 0
)

#Scaling (need for LR and kNN)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
In [7]: #Logistic Regression
from sklearn.linear_model import LogisticRegression
```

```

lr = LogisticRegression()
lr.fit(X_train_scaled, y_train)
y_pred_lr = lr.predict(X_test_scaled)

print("Logistic Regression")
print("Accuracy: ", accuracy_score(y_test, y_pred_lr))
print("Confusion Matrix\n" , confusion_matrix(y_test, y_pred_lr))

```

Logistic Regression

Accuracy: 0.85

Confusion Matrix

```

[[25  7]
 [ 2 26]]

```

```

In [16]: #Decision Tree
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier(max_depth=4)

from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
y_pred_knn = knn.predict(X_test_scaled)

print("kNN")
print("Accuracy: ", accuracy_score(y_test, y_pred_knn))
print("Confusion Matrix\n" , confusion_matrix(y_test, y_pred_knn))

```

kNN

Accuracy: 0.9

Confusion Matrix

```

[[27  5]
 [ 1 27]]

```

```

In [14]: def plot_boundary(model, scaled, title):
    h = 0.02

    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

    xx, yy = np.meshgrid(
        np.arange(x_min, x_max, h),
        np.arange(y_min, y_max, h))

    grid = np.c_[xx.ravel(), yy.ravel()]

    if scaled:
        grid = scaler.transform(grid)

    z = model.predict(grid)
    z = z.reshape(xx.shape)

    plt.contourf(xx, yy, z, alpha=0.3)
    plt.scatter(X[:, 0], X[:, 1], c=y)
    plt.title(title)
    plt.show()

plot_boundary(lr, True, "Logistic Regression Boundary")
plot_boundary(dt, True, "Decision Tree Boundary")
plot_boundary(knn, True, "kNN Boundary")

```

Logistic Regression Boundary

